Module 19: Doubly Linked List Insertion

1. Current Progress Analysis

You are now at **Module 19**.

Before this module, you already learned the main ideas needed to understand doubly linked-list insertion.

Previous Topic	Why It Matters in Module 19
Singly linked lists	You know that a node can store data and a link to another node.
Traversal	You know how to move through a list using `p = p->next`.
Insertion and deletion	You know that linked lists change links instead of shifting array values.
Circular singly linked lists	You know that some lists do not end normally at `NULL`.
Doubly linked-list foundations	You know that a doubly linked-list node has `prev`, `data`, and `next`.

In Module 18, you learned the structure of a doubly linked-list node:

```
struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};
```

A doubly linked list looks like this:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

This means:

- The first node has no previous node, so its `prev` is `NULL`.
- The last node has no next node, so its `next` is `NULL`.
- Every middle node has both a previous node and a next node.

Module 19 teaches how to **insert a new node into a doubly linked list** while keeping both directions correct.

---

2. Big Idea of Module 19

In a singly linked list, each node has only one link:

```
[data | next]
```

So insertion mostly changes the `next` links.

Example singly linked-list insertion pattern:

```
t->next = p->next;
p->next = t;
```

But in a doubly linked list, each node has two links:

```
[prev | data | next]
```

So insertion must update:

```
forward links + backward links
```

Example list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

If we insert `15` between `10` and `20`, the result should be:

```
NULL <- 10 <-> 15 <-> 20 <-> 30 -> NULL
```

The new node `15` must point in both directions:

```
15->prev = 10
15->next = 20
```

The surrounding nodes must also point to the new node:

```
10->next = 15
20->prev = 15
```

That is the main idea of Module 19:

> When inserting into a doubly linked list, connect the new node in both directions.

---

3. Doubly Linked List Reminder

A doubly linked list has nodes like this:

```
[prev | data | next]
```

Each field has a job.

Field	Meaning
`prev`	Address of the previous node
`data`	Value stored in the node
`next`	Address of the next node

Example:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

For node `20`:

```
20->prev points to 10
20->next points to 30
```

For node `10`:

```
10->prev == NULL
10->next points to 20
```

For node `30`:

```
30->prev points to 20
30->next == NULL
```

---

4. Meaning of `first`

The pointer `first` stores the address of the first node.

Example:

```
first
 |
 v
NULL <- 10 <-> 20 <-> 30 -> NULL
```

If the list is empty:

```
first == NULL
```

Important beginner rule:

> Do not move `first` unless you intentionally want to change the beginning of the list.

For normal movement through the list, use a temporary pointer like `p`.

Example:

```
struct Node *p = first;
```

Then move `p`:

```
p = p->next;
```

This keeps `first` safe.

---

5. Index Convention for Insertion

Module 19 uses this insertion index convention:

Index	Meaning
`0`	Insert before the first node
`1`	Insert after the first node
`2`	Insert after the second node
`n`	Insert after the last node in a list of length `n`

Example list:

```
10 <-> 20 <-> 30
```

This list has 3 nodes.

Valid insertion indexes are:

```
0, 1, 2, 3
```

Examples:

Call	Meaning	Result
`Insert(0, 5)`	Insert before `10`	`5 <-> 10 <-> 20 <-> 30`
`Insert(1, 15)`	Insert after `10`	`10 <-> 15 <-> 20 <-> 30`
`Insert(2, 25)`	Insert after `20`	`10 <-> 20 <-> 25 <-> 30`
`Insert(3, 40)`	Insert after `30`	`10 <-> 20 <-> 30 <-> 40`

Invalid indexes for this 3-node list:

```
-1, 4, 100
```

If a list has `n` nodes, valid insertion indexes are:

```
0 to n
```

So the validation rule is:

```
if (index < 0 || index > length(first)) {
    return;
}
```

Meaning:

- If `index < 0`, the index is too small.
- If `index > length(first)`, the index is too large.
- In both cases, the function should stop without changing the list.

---

6. Why Insertion in a Doubly Linked List Needs Extra Care

In a singly linked list, we usually only update `next`.

In a doubly linked list, we must update both:

```
prev links
next links
```

If we update only the `next` links, forward traversal may work, but backward traversal will be broken.

Example wrong result:

```
NULL <- 10 -> 15 -> 20 -> NULL
```

This may look connected going forward, but if `20->prev` does not point back to `15`, the doubly linked list is not correct.

Correct result:

```
NULL <- 10 <-> 15 <-> 20 -> NULL
```

The list must work in both directions:

Forward:

```
10 -> 15 -> 20
```

Backward:

```
20 -> 15 -> 10
```

---

7. Main Insertion Cases

Doubly linked-list insertion has three main cases:

1. Insert at the head.
2. Insert in the middle.
3. Insert at the end.

Each case uses the same basic idea, but the exact links are slightly different.

Case 1: Insert at the Head

8. Head Insertion Meaning

Head insertion means inserting before the current first node.

Original list:

```
first
 |
 v
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Insert `5` at index `0`.

Expected result:

```
first
 |
 v
NULL <- 5 <-> 10 <-> 20 <-> 30 -> NULL
```

The new node becomes the first node.

---

9. Head Insertion Steps

Create new node `t`.

Then connect it like this:

```
t->prev = NULL;
t->next = first;

if (first != NULL) {
    first->prev = t;
}

first = t;
```

Meaning of each line:

Code	Meaning
`t->prev = NULL;`	The new node has no previous node because it will become the first node.
`t->next = first;`	The new node points forward to the old first node.
`if (first != NULL)`	Check whether the old first node exists.
`first->prev = t;`	The old first node points backward to the new node.
`first = t;`	The external `first` pointer now points to the new node.

---

10. Why We Check `first != NULL`

This line is unsafe if the list is empty:

```
first->prev = t;
```

Why?

If the list is empty:

```
first == NULL
```

Then this would mean:

```
NULL->prev = t
```

That is invalid because `NULL` is not a real node.

So we write:

```
if (first != NULL) {
    first->prev = t;
}
```

This handles both cases:

Situation	What Happens
Empty list	There is no old first node, so skip `first->prev = t`.
Non-empty list	Update the old first node's `prev` pointer.

---

11. Dry Run: Insert at Head

Original list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Call:

```
Insert(0, 5);
```

Step 1: Create new node `t`.

```
[ ? | 5 | ? ]
```

Step 2: Set `t->prev = NULL`.

```
NULL <- 5
```

Step 3: Set `t->next = first`.

```
NULL <- 5 -> 10 <-> 20 <-> 30 -> NULL
```

Step 4: Set `first->prev = t`.

```
NULL <- 5 <-> 10 <-> 20 <-> 30 -> NULL
```

Step 5: Set `first = t`.

```
first
 |
 v
NULL <- 5 <-> 10 <-> 20 <-> 30 -> NULL
```

Final list:

```
5 10 20 30
```

Case 2: Insert in the Middle

12. Middle Insertion Meaning

Middle insertion means inserting between two existing nodes.

Original list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Insert `15` after `10`.

Expected result:

```
NULL <- 10 <-> 15 <-> 20 <-> 30 -> NULL
```

---

13. Pointer Position for Middle Insertion

For non-head insertion, we use pointer `p`.

`p` must point to the node after which we want to insert.

Example:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
         ^
         |
         p
```

Here, `p` points to `10`.

The new node `15` must be inserted after `p` and before `p->next`.

So:

```
p points to 10
p->next points to 20
```

---

14. Middle Insertion Link Pattern

The correct middle insertion pattern is:

```
t->prev = p;
t->next = p->next;
p->next->prev = t;
p->next = t;
```

Meaning:

Code	Meaning
`t->prev = p;`	New node points backward to the node before it.
`t->next = p->next;`	New node points forward to the node after it.
`p->next->prev = t;`	The next node points backward to the new node.
`p->next = t;`	The previous node points forward to the new node.

This is called a **4-link insertion** because four pointer fields are updated.

---

15. Dry Run: Insert in the Middle

Original list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Call:

```
Insert(1, 15);
```

Index `1` means insert after the first node.

So `p` points to `10`.

Before insertion:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
         ^
         |
         p
```

Step 1: Create node `t` with data `15`.

```
[ ? | 15 | ? ]
```

Step 2: Set `t->prev = p`.

```
10 <- 15
```

Step 3: Set `t->next = p->next`.

Since `p->next` is `20`, this means:

```
15 -> 20
```

Now the new node knows both sides:

```
10 <- 15 -> 20
```

Step 4: Set `p->next->prev = t`.

This makes node `20` point backward to `15`.

```
15 <- 20
```

Step 5: Set `p->next = t`.

This makes node `10` point forward to `15`.

Final result:

```
NULL <- 10 <-> 15 <-> 20 <-> 30 -> NULL
```

---

16. Why the Order of Link Changes Matters

The order is important because we do not want to lose access to the rest of the list.

Correct order:

```
t->prev = p;
t->next = p->next;
p->next->prev = t;
p->next = t;
```

The important idea is:

> Save the old next link before overwriting it.

This line saves the node after `p`:

```
t->next = p->next;
```

Only after that do we change:

```
p->next = t;
```

If we overwrite `p->next` too early, we can lose the connection to the old next node.

Case 3: Insert at the End

17. End Insertion Meaning

End insertion means inserting after the last node.

Original list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Insert `40` at the end.

Expected result:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL
```

---

18. Pointer Position for End Insertion

For end insertion, `p` points to the last node.

```
NULL <- 10 <-> 20 <-> 30 -> NULL
                        ^
                        |
                        p
```

Here:

```
p->next == NULL
```

That means there is no node after `p`.

---

19. End Insertion Link Pattern

The end insertion pattern is:

```
t->prev = p;
t->next = NULL;
p->next = t;
```

Meaning:

Code	Meaning
`t->prev = p;`	New node points backward to the old last node.
`t->next = NULL;`	New node has no next node because it is now the last node.
`p->next = t;`	Old last node points forward to the new node.

This is called a **3-link insertion** because only three pointer fields are updated.

---

20. Why End Insertion Uses Only 3 Links

Middle insertion uses this line:

```
p->next->prev = t;
```

But at the end of the list:

```
p->next == NULL
```

So this would be invalid:

```
p->next->prev = t;
```

because it would mean:

```
NULL->prev = t
```

That is not allowed.

So for end insertion, we do not update `p->next->prev`.

Instead, we use:

```
t->prev = p;
t->next = NULL;
p->next = t;
```

---

21. Dry Run: Insert at End

Original list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Call:

```
Insert(3, 40);
```

Index `3` means insert after the third node.

So `p` points to `30`.

Before insertion:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
                        ^
                        |
                        p
```

Step 1: Create node `t` with data `40`.

```
[ ? | 40 | ? ]
```

Step 2: Set `t->prev = p`.

```
30 <- 40
```

Step 3: Set `t->next = NULL`.

```
40 -> NULL
```

Step 4: Set `p->next = t`.

```
30 -> 40
```

Final result:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL
```

---

22. 3 Links vs 4 Links

This is one of the most important ideas in Module 19.

Middle Insertion: 4 Links

When inserting in the middle, there is a node after the insertion point.

Example:

```
10 <-> 20
```

Insert `15` between them.

We must update four links:

```
t->prev = p;
t->next = p->next;
p->next->prev = t;
p->next = t;
```

Why four?

Link	Purpose
`t->prev = p`	New node points backward.
`t->next = p->next`	New node points forward.

Link	Purpose
`p->next->prev = t`	Next node points backward to the new node.
`p->next = t`	Previous node points forward to the new node.

End Insertion: 3 Links

When inserting at the end, there is no node after the insertion point.

Example:

```
30 -> NULL
```

Insert `40` after `30`.

We update only three links:

```
t->prev = p;
t->next = NULL;
p->next = t;
```

Why only three?

Because there is no next node whose `prev` must be updated.

---

23. The Most Important Safety Check

This line is dangerous if `p` points to the last node:

```
p->next->prev = t;
```

Why?

If `p` is the last node, then:

```
p->next == NULL
```

So the code becomes logically like this:

```
NULL->prev = t
```

That is invalid.

Correct safe version:

```
if (p->next != NULL) {
    p->next->prev = t;
}
```

This means:

> Only update the next node's `prev` pointer if the next node actually exists.

This single check allows the same code to handle both middle insertion and end insertion.

---

24. Complete Insert Function

The full insertion function is:

```
void Insert(int index, int x) {
    struct Node *t;
    struct Node *p;

    if (index < 0 || index > length(first)) {
        return;
    }

    t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;

    if (index == 0) {
        t->prev = NULL;
        t->next = first;

        if (first != NULL) {
            first->prev = t;
        }

        first = t;
    } else {
        p = first;
```

```
        for (int i = 0; i < index - 1; i++) {
            p = p->next;
        }

        t->prev = p;
        t->next = p->next;

        if (p->next != NULL) {
            p->next->prev = t;
        }

        p->next = t;
    }
}
```

---

25. Explanation of the Insert Function

Function Header

```
void Insert(int index, int x)
```

Meaning:

Part	Meaning
`void`	The function does not return a value.
`Insert`	Function name.
`index`	Position where the new node should be inserted.
`x`	Value to store inside the new node.

Pointer Variables

```
struct Node *t;
struct Node *p;
```

Meaning:

Pointer	Role
`t`	New node to be inserted.
`p`	Temporary pointer used to reach the insertion position.

Index Validation

```
if (index < 0 || index > length(first)) {
    return;
}
```

This checks whether the insertion index is valid.

If the index is invalid, the function stops.

For a list with `n` nodes, valid indexes are:

```
0 to n
```

Create the New Node

```
t = (struct Node *)malloc(sizeof(struct Node));
t->data = x;
```

This creates a new node in heap memory and stores `x` inside it.

At this point, the new node is not connected yet.

Head Insertion Case

```
if (index == 0) {
    t->prev = NULL;
    t->next = first;

    if (first != NULL) {
        first->prev = t;
    }
```

```
    first = t;
}
```

This inserts the new node before the current first node.

It also works if the list is empty.

Non-Head Insertion Case

```
else {
    p = first;

    for (int i = 0; i < index - 1; i++) {
        p = p->next;
    }

    t->prev = p;
    t->next = p->next;

    if (p->next != NULL) {
        p->next->prev = t;
    }

    p->next = t;
}
```

This handles insertion after an existing node.

The loop moves `p` to the node after which insertion should happen.

Then the new node is connected using both `prev` and `next` links.

---

26. How the Loop Finds the Correct Node

For non-head insertion, we use:

```
p = first;

for (int i = 0; i < index - 1; i++) {
```

```
    p = p->next;
}
```

Why `index - 1`?

Because `p` starts at the first node.

Example list:

```
10 <-> 20 <-> 30 <-> 40
```

Suppose we call:

```
Insert(3, 35);
```

Index `3` means insert after the third node.

The third node is `30`.

Start:

```
p points to 10
```

Movements:

Movement	p Points To
Start	`10`
Move 1	`20`
Move 2	`30`

For index `3`, we move `2` times.

That is why the loop runs:

```
index - 1
```

times.

---

27. Full C Program for Module 19

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};

struct Node *first = NULL;

int length(struct Node *p) {
    int len = 0;

    while (p != NULL) {
        len++;
        p = p->next;
    }

    return len;
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
    printf("\n");
}

void create(int A[], int n) {
    if (n <= 0) {
        first = NULL;
        return;
    }

    struct Node *t;
    struct Node *last;

    first = (struct Node *)malloc(sizeof(struct Node));
    first->prev = NULL;
    first->data = A[0];
    first->next = NULL;
```

```
    last = first;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->prev = last;
        t->data = A[i];
        t->next = NULL;

        last->next = t;
        last = t;
    }
}

void Insert(int index, int x) {
    struct Node *t;
    struct Node *p;

    if (index < 0 || index > length(first)) {
        return;
    }

    t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;

    if (index == 0) {
        t->prev = NULL;
        t->next = first;

        if (first != NULL) {
            first->prev = t;
        }

        first = t;
    } else {
        p = first;

        for (int i = 0; i < index - 1; i++) {
            p = p->next;
        }

        t->prev = p;
        t->next = p->next;

        if (p->next != NULL) {
            p->next->prev = t;
        }

        p->next = t;
```

```
    }
}

int main() {
    int A[] = {10, 20, 30};

    create(A, 3);

    printf("Original list: ");
    display(first);

    Insert(0, 5);
    printf("After inserting 5 at index 0: ");
    display(first);

    Insert(2, 15);
    printf("After inserting 15 at index 2: ");
    display(first);

    Insert(5, 40);
    printf("After inserting 40 at end: ");
    display(first);

    return 0;
}
```

Possible output:

```
Original list: 10 20 30
After inserting 5 at index 0: 5 10 20 30
After inserting 15 at index 2: 5 10 15 20 30
After inserting 40 at end: 5 10 15 20 30 40
```

---

28. Important Note About the Example `Insert(5, 40)`

After these operations:

```
Insert(0, 5);
Insert(2, 15);
```

The list becomes:

```
5 <-> 10 <-> 15 <-> 20 <-> 30
```

This list has 5 nodes.

So valid insertion indexes are:

```
0, 1, 2, 3, 4, 5
```

Therefore:

```
Insert(5, 40);
```

is valid.

It inserts after the fifth node, which is `30`.

Final list:

```
5 <-> 10 <-> 15 <-> 20 <-> 30 <-> 40
```

---

29. Complexity Analysis

Insert at Index 0

Head insertion does not need traversal.

It only changes a fixed number of links.

So the time complexity is:

```
O(1)
```

Insert After a Known Node `p`

If we already have pointer `p`, insertion also only changes a fixed number of links.

So the time complexity is:

```
O(1)
```

Insert by Index in General

If we are given only an index, we must move from `first` to the correct position.

This movement may take time:

```
p = p->next;
```

In the worst case, we may move through many nodes.

So the time complexity is:

```
O(n)
```

Extra Space Complexity

The function uses only:

- one new node,
- pointer `t`,
- pointer `p`,
- a few simple variables.

Extra working space is:

```
O(1)
```

The new node is part of the data structure, not temporary extra working space.

---

30. Common Beginner Mistakes

Mistake 1: Forgetting to Update `prev`

Wrong idea:

```
t->next = p->next;
p->next = t;
```

This is only singly linked-list thinking.

In a doubly linked list, we must also update `prev` links.

Correct idea:

```
t->prev = p;
t->next = p->next;

if (p->next != NULL) {
    p->next->prev = t;
}

p->next = t;
```

Mistake 2: Using `p->next->prev` Without Checking `p->next`

Wrong:

```
p->next->prev = t;
```

This crashes or behaves incorrectly if `p` is the last node.

Correct:

```
if (p->next != NULL) {
    p->next->prev = t;
}
```

Mistake 3: Not Updating `first` During Head Insertion

Wrong:

```
t->next = first;
first->prev = t;
```

This connects the new node, but `first` may still point to the old first node.

Correct:

```
t->prev = NULL;
t->next = first;

if (first != NULL) {
    first->prev = t;
}

first = t;
```

Mistake 4: Invalid Index Not Checked

Wrong:

```
Insert(-1, 50);
Insert(100, 50);
```

These should not be allowed.

Correct validation:

```
if (index < 0 || index > length(first)) {
    return;
}
```

---

31. Beginner Rule for Doubly Linked-List Insertion

Before changing links, ask four questions:

```
1. Who is before the new node?
2. Who is after the new node?
```

```
3. Does the after-node exist?
4. Does first need to change?
```

Then connect the new node in both directions.

---

32. Core Patterns to Memorize

Head Insertion

```
t->prev = NULL;
t->next = first;

if (first != NULL) {
    first->prev = t;
}

first = t;
```

Middle or End Insertion

```
t->prev = p;
t->next = p->next;

if (p->next != NULL) {
    p->next->prev = t;
}

p->next = t;
```

This one pattern works for both middle and end insertion because of the safety check:

```
if (p->next != NULL)
```

---

33. Final Summary

Module 19 teaches insertion into a doubly linked list.

The main difference from singly linked-list insertion is that a doubly linked list has two directions.

So insertion must maintain:

```
prev links + next links
```

Important points:

- A doubly linked-list node has `prev`, `data`, and `next`.
- Index `0` means insert before the first node.
- Index `n` means insert after the last node in a list of length `n`.
- Head insertion must update `first`.
- Middle insertion usually updates 4 links.
- End insertion updates only 3 links.
- Never use `p->next->prev` unless `p->next != NULL`.
- General insertion by index is `O(n)` because we may need traversal.
- Actual link changing after reaching the position is `O(1)`.

The most important code pattern is:

```
t->prev = p;
t->next = p->next;

if (p->next != NULL) {
    p->next->prev = t;
}

p->next = t;
```

If you understand why this works, you understand the core of Module 19.

---

34. Practice Task

Dry-run this operation by hand:

Original list:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL
```

Call:

```
Insert(2, 25);
```

Questions:

1. Which node should `p` point to before insertion?
2. What should `t->prev` point to?
3. What should `t->next` point to?
4. Does `p->next->prev` need to be updated?
5. What is the final list?

Expected final list:

```
NULL <- 10 <-> 20 <-> 25 <-> 30 <-> 40 -> NULL
```